

„ZPR PWr – Zintegrowany Program Rozwoju Politechniki Wrocławskiej”

Game testing

Marek Kopel

Lecture

1. **Silniki gier. Wprowadzenie do Unity. Pierwsza gra 2D.**
2. **Mechanika 2D, animacje.**
3. **Oświetlenie, tekstury, materiały. Pierwsza gra 3D.**
4. Narzędzia wspierające, np. Blender.
5. **Zapis/odczyt danych, komunikacja sieciowa.**
6. **Modelowanie terenu. Generowanie terenu.**
7. **Sztuczna inteligencja w grach.**
8. Virtual Reality, wsparcie VR w Unity.
9. **Optymalizacje w Unity.**
10. Historia i klasyfikacja gier.
11. Prototypowanie gier. GDD.
12. Projektowanie poziomów gier.
13. Projektowanie gier dla różnych platform.
14. Testowanie gier. Test.
15. Test.

Role: tester

analyzes video games to document software defects

→ QA - quality assurance

highly **technical** ← requires **computing** expertise and **analytic** competence

? game falls within the proposed design ?

→ testing of all features, compatibility, localization, etc.

necessary throughout the whole development process

← often utilized only at the end (expensive) → RC - release candidate

+ public beta-testing

bugs severity:

A bugs are critical bugs (e.g. crash the game)

B bugs are essential (require attention), but game may still be playable

C bugs are small and obscure, often recommendation rather than problems

Testing

quality assurance (QA) plays a **significant** role

once anything is **playable** → testers start
(may be one level or subset of the game software)

early on ← relatively small amount of time
testers may work on several games at once

before launch (**crunch** time) → many testers full-time (and often with **overtime**)

test **new** features + **existing** ones (“**regression** testing”)

modern, complex games: small changes → **catastrophic** consequences

[Flappy Mario](#)

Testing

Regression testing: often overlooked

- feature is implemented and tested $\Rightarrow$ "working"
- features added late $\Rightarrow$ prioritized
- regression testing $\rightarrow$ increasingly expensive (#features increases)
 $\rightarrow$ often not scheduled correctly

even when fully tested:

game shipped with bugs

- $\rightarrow$ customers dissatisfaction
- $\rightarrow$ sales failure
- $\rightarrow$ patches releases (fix the bugs)

Test

“Automated code that invokes a system and checks behaviour.”

based on

<https://www.slideshare.net/tenpn/test-driven-development-25476304>

Unit Test

“Checks a single assumption about the behavior of one system.”

Integration Test

“Checks multiple systems
are correctly connected”

Unit Test != Integration Test

Only fails when target
assumption fails

Fails when one of multiple
implicit assumptions fail

Readable

Qualities of a Unit Test

Maintainable

Trustworthy

[Test]

```
void UpdatePath_OneActionChild_AddsSelAndChildToPath() {
```

```
    var actionNode = FakeAction.CreateStub();
```

```
    m_selNode.AddChild(actionNode);
```

```
    m_selNode.UpdatePath(m_path, NodeResult.Fails, null);
```

```
    var desiredPath = new ITreeNode[] {
```

```
        m_selNode, actionNode };
```

```
    Assert.IsEqualSequence(m_path, desiredPath);
```

```
}
```

TDD

- 1: Write a unit test
- 2: Watch it fail
- 3: Make it pass *simply*
- 4: Refactor
- 5: Goto 1

SCIENCE!

- 15-30% higher code “quality”
- 10-20% longer development time

MonoBehaviours are
un-unit-testable



NON-TRIVIAL

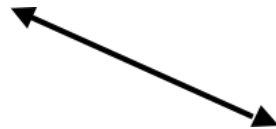
No post-
compile hook

Lots of alt-
tabbing

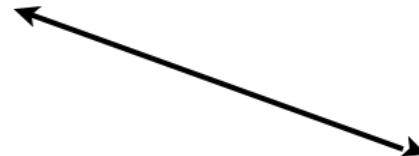
Model View Controller



MonoBehaviour **View**



Struct data **Model**



Game code **Controller**

Model View Controller



class GunController

TDD

struct GunModel

class Gun : MonoBehaviour

class InitialGunSetupView : MonoBehaviour

class AmmoView : MonoBehaviour

Model View Controller



```
class Gun : MonoBehaviour {  
    public GunModel Model { get; private set; }
```

```
    void Awake( ) {  
        Model = new GunModel( );  
        m_controller = new GunController(Model);
```

```
class AmmoView : MonoBehaviour {  
    void Awake( ) {  
        m_model = GetComponent<Gun>( ).GunModel;
```

Dependency Injection



```
public void GunController(GunModel model, IWorldPhysics physics) {  
    Physics = physics;  
    Model = model;  
}
```



- Unit tests test only one thing
- Readable, maintainable, trustable
- SCIENCE says TDD is selective
- Write test, see fail, make pass, refactor
- Model-View-Controller and Dependency Injection make TDD possible in Unity

well, actually... Unity Test Framework!

<https://docs.unity3d.com/Manual/testing-editor-test-runner.html>

Lessons learned (2020)

[Writing Tests with the Unity Test Framework | PPT](#)

Why write automated tests in general?

Tests are about getting feedback and managing risk.

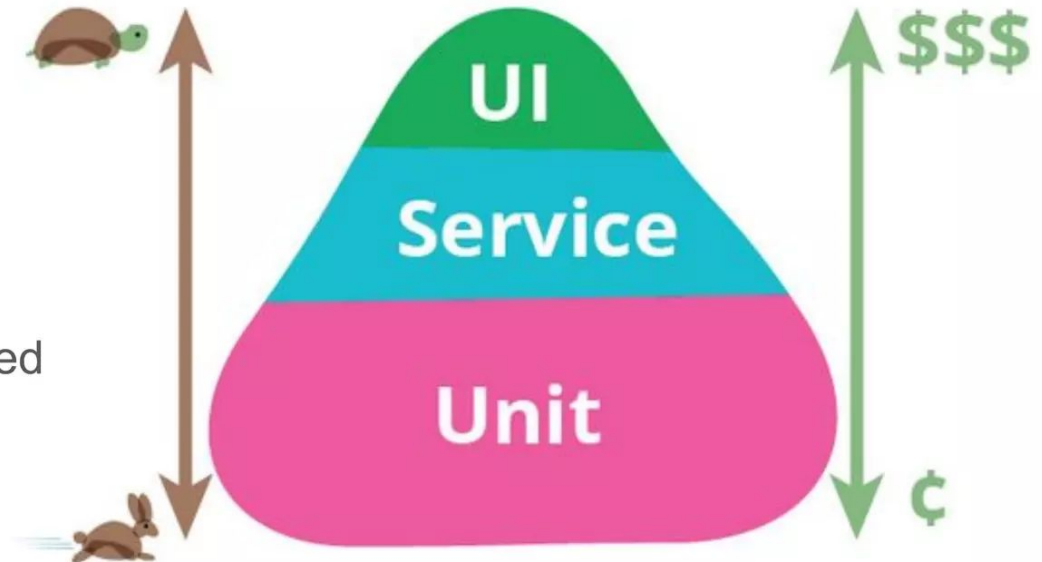
A big part is the (business) logic and is usually automatic testable.

Benefits of automated tests:

- Get feedback during development if your ideas work (TDD)
- Keep features working if code is changed (Regressions)
 - Allow refactoring
 - Return on Investment (ROI)
- Volume (thousands of cases) & scale testing to other platforms, devices, etc.
- Running tests 24/7 (“addicted to feedback”)

Unit- vs. Integrated Tests

- System tests are easier to write (and often the only ones possible) for user interfaces.
- UI tests are much slower, feedback takes longer, tests are run rarely.
- Integrated tests are not aligned with TDD process and have less benefits.



Why write the test first?

When using Test Driven Development you write your test **before** you write the production code.

Pros of TDD:

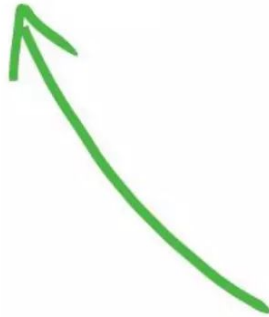
- Faster feedback of your ideas
- We develop inherently testable code
- We design the production code interface from the viewpoint of calling code
 - encourages modular design with few dependencies

Cons: Hard to learn. Doesn't work well in all cases.

Write a
failing
test

Make the
test pass

Refactor



What about writing tests for user interfaces?

Graphics and user interface work is a lot of “fiddling” with pixels and coordinates. You must “massage” the code into place. It needs a very rapid cycle of edit-and-run. (Uncle Bob)

Most people agree that we are not writing automated tests for

- styles
- colours
- pixel perfect positions
- maybe hard coded text displayed

What about testing in the 3D world?

Is it even possible?

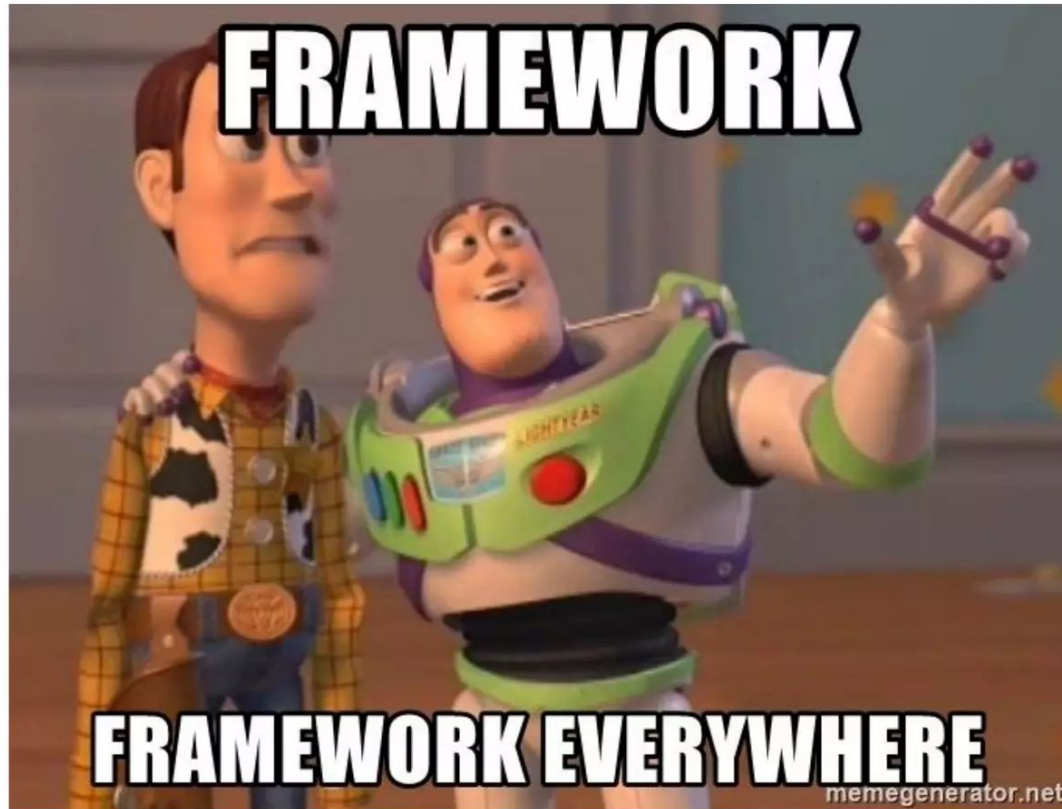
We would like to have the benefits of automated tests.

What about TDD?

We tried a sample project, approx. 30 hours pair programming.

This talk is about what we learned...

Btw: We don't want to test the graphics engine or the GPU driver.



Unity Test Framework

Based on NUnit

Basic test method

```
[Test]
public void HelloWorldTest()
{
    // Act
    var message = new Message();

    // Assert
    Assert.That(message.Text, Is.EqualTo("Hello World"));
}
```

Based on NUnit

Arrange - Act - Assert

```
[Test]
public void HelloWorldTest()
{
    // Arrange
    Environment.PrepareStuff();

    // Act
    var message = new Message();

    // Assert
    Assert.That(message.Text, Is.EqualTo("Hello World"));
}
```

Based on NUnit

Fixtures

```
[SetUp]
public void Init() { /* ... */ }

[TearDown]
public void Cleanup() { /* ... */ }

[Test]
public void HelloWorldTest()
{
    // Act
    var message = new Message();

    // Assert
    Assert.That(message.Text, Is.EqualTo("Hello World"));
}
```

Based on NUnit

One time Fixtures

```
[OneTimeSetUp]
public void Init() { /* ... */ }

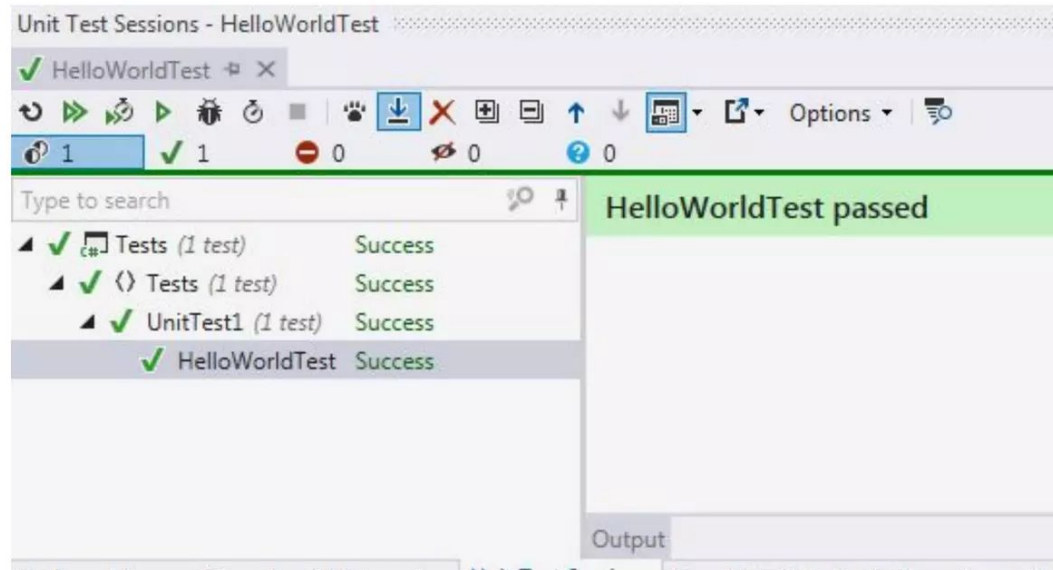
[OneTimeTearDown]
public void Cleanup() { /* ... */ }

[Test]
public void HelloWorldTest()
{
    // Act
    var message = new Message();

    // Assert
    Assert.That(message.Text, Is.EqualTo("Hello World"));
}
```

Based on NUnit

Test Runner



Unity Test Framework

- Install via the Unity Package Manager
- Play-mode tests
 - ⇒ load scenes and assert run-time behaviour
- Test builds
 - ⇒ run tests on different platforms (mobile, consoles, WebAssembly)

<https://docs.unity3d.com/Packages/com.unity.test-framework@1.1>

Play-mode vs. Edit-mode Test

Play-mode Tests

- run **standalone** in a Player or inside the Editor

Edit-mode Tests

- only run in the Unity Editor
- and **have access to the Editor code** in addition to the game code
⇒ with Edit Mode tests it is possible to test any of your Editor extensions

Play-mode vs. Unit Test

Prefer unit tests, i.e. `NUnit Test` attribute.

Use unit tests instead of the `UnityTest` attribute, unless you need to **skip a frame** or **wait for a certain amount of time** (that is unless you need something of the Unity engine).

In Play Mode, the `UnityTest` attribute runs as a coroutine.

Avoid (slow) Integration Tests

It's tempting to often test with a Integration Test, but it comes at a cost.

Pattern:

If it's unclear if a Unit Test is possible,
start with an Integration Test first
and try to refactor to a unit test later.



Scene Loading

Zenject (Dependency Injection framework) offers a `SceneTestFixture`

```
public class TestSceneStartup : SceneTestFixture
{
    [UnityTest]
    public IEnumerator TestSpaceFighter()
    {
        yield return LoadScene("SpaceFighter");

        // Wait a few seconds to ensure the scene starts correctly
        yield return new WaitForSeconds(2.0f);
    }
}
```



Query Object by ID

We used an `Identifier` script to query the `GameObject` from the test. This way, failing tests are avoided when the objects are renamed.

```
public static class Find
{
    public static GameObject SingleObjectById(string objectId)
    {
        var gameObjects = Object.FindObjectsOfType<Identifier>()
            .Where(identifier => identifier.id == objectId)
            .Select(identifier => identifier.gameObject)
            .ToList();
        Assert.That(gameObjects.Count, Is.EqualTo(1), $"Object {objectId} not found");
        return gameObjects[0];
    }
}
```

Linear search only works for scenes with not many objects.



Slow tests

- Make (real time) tests faster $\Rightarrow$ tweak `Time.timeScale`
- Make your tests faster than real-time,
but make sure to not execute “unnecessary” things first.
- Watch out for stability...
Maybe have a nightly-build with real-time scale as well?



Waiting for something to be ready

- We created helper coroutines for better readability.

```
yield return Given.Scene("TestEnvironmentScene");
```

- **Recommendation:** Keep the actual test method clean and readable, future maintainers (or future you) will thank you 🙏



Input Testing

The (new) Input System allows to simulate user input.

```
[Test]
public void TestGamepad()
{
    var gamepad = InputSystem.AddDevice<Gamepad>();

    Press(gamepad.buttonSouth);
    Set(gamepad.leftStick, new Vector2(0.123f, 0.234f));
}
```

<https://docs.unity3d.com/Packages/com.unity.inputsystem@1.0/manual/Testing.html>



Comparing decimal numbers

```
[Test]
public void VerifyThat_TwoFloatsAreEqual()
{
    var comparer = new FloatEqualityComparer(10e-6f);
    var actual = -0.00009f;
    var expected = 0.00009f;

    Assert.That(actual, Is.EqualTo(expected).Using(comparer));
}
```

allowed error



Comparing Vectors

```
[Test]
public void VerifyThat_TwoVector3ObjectsAreEqual()
{
    // Custom error 10e-6f
    var actual = new Vector3(10e-8f, 10e-8f, 10e-8f);
    var expected = new Vector3(0f, 0f, 0f);
    var comparer = new Vector3EqualityComparer(10e-6f);

    Assert.That(actual, Is.EqualTo(expected).Using(comparer));
}
```

allowed error



Parameterized Test

Reduce duplication and increase test coverage.

```
[Test]
public void MyTest([Values(1, 2, 3)] int x, [Values("A", "B")] string s)
{
    /* ... */
}
```



```
MyTest(1, "A")
MyTest(1, "B")
MyTest(2, "A")
MyTest(2, "B")
MyTest(3, "A")
MyTest(3, "B")
```



Parameterized Test

Recommendation:

Parameters should only test one “dimension”.

Don’t merge different “requirements” because it’s easy to do. It will make analyzing a failing test more difficult.

Balance test expressiveness vs code reuse.



Conclusion

Automated tests pay off in the future. They are a tool to manage risk.

Testing 3D components in Unity is possible.

Some of the 3D stuff was easy to test.

Not all things (on the UI) can be tested nor need to be tested.

Without experience it is hard to tell where tests are most beneficial.

You need to get started now!

Unity Test Framework - DEMO

The image shows a screenshot of the Unity Test Runner interface. The interface is divided into two main sections: a left sidebar and a main content area.

Left Sidebar:

- Header: Test Runner
- Buttons: PlayMode, EditMode
- Buttons: Run All, Run Selected, Rerun Failed
- Search bar: q
- Dropdown: Nothing
- Summary: 1 green checkmark, 1 red circle with a slash, 0 empty circle.
- Test Results (expanded):
 - ✓ TDD
 - ✓ EditMode.dll
 - ✓ DirectionTests
 - ✓ DirectionTestsSin
 - ✓ DirectionTestsWith

Main Content Area:

- Header: Test Runner
- Buttons: PlayMode, EditMode
- Buttons: Run All, Run Selected, Rerun Failed, Clear Results
- Search bar: q
- Dropdown: Nothing
- Summary: 1 green checkmark, 1 red circle with a slash, 0 empty circle.
- Test Results (expanded):
 - ✗ TDD
 - ✗ EditMode.dll
 - ✗ DirectionTests
 - ✓ North
 - ✗ South (highlighted in blue)

Details for the failed test 'South':

- South (0,006s)
-
- Expected: 1
- But was: 0
-
- at DirectionTests.South () [0x00000] in C:\Users\marek\

Unity Test Framework - DEMO

Unit Tests in Unity

<https://www.patreon.com/posts/37986282>

1. new Test folder
2. Window > General > Test Runner
3. Create Edit-mode Test Assembly folder (name: EditMode)
4. Create Test Script in current folder (name: DirectionTests)
5. To access Script folder from tests
Add: Create > Assembly Definitions (name: Scripts)
6. And add Assembly Definition Reference to it in EditMode file (thru inspector)
<https://forum.unity.com/threads/text-mesh-pro-type-or-namespace-issue.810012/>
7. Write test method preceded with [Test] attribute in DirectionTests file.